The role of orientation numbers in the `atlas` unitarity algorithm

June 23, 2026

1 The question

The `atlas` algorithm for deciding whether an irreducible representation $\pi = J(\gamma)$ is unitary proceeds in two stages:

(1) compute the signature of the *c-form* on π;

(2) *convert* that signature into the signature of the genuine invariant Hermitian form on π.

The representation is unitary iff the resulting Hermitian form is definite (`is_pure` in the code). The question is: where do orientation numbers enter? Do they belong to stage (1), the c-form computation, or only to stage (2), the Hermitian conversion?

Short answer. Orientation numbers belong to stage (1). They appear in the passage from *standard* modules to the *irreducible* module *inside the c-form computation.* They do *not* appear in the c-to-Hermitian conversion of stage (2); that conversion is governed by a different invariant μ (built from $\dim(\mathfrak{u} \cap \mathfrak{p})$ and the value of the lowest K-type). So the naive guess — "orientation numbers are the thing that turns the c-form into the Hermitian form" — is, perhaps surprisingly, *not* how the algorithm is organized.

2 What an orientation number is

The signature characters live in the ring of *split integers* $\mathbb{Z}[s]/(s^2 - 1)$: the unit s records a sign, i.e. it interchanges the positive and negative parts of a signature (p, q).

The orientation number $\mathrm{or}(\gamma)$ of a parameter $\gamma = (x, \lambda, \nu)$ is a nonnegative integer attached to γ. It is computed by the built-in `orientation_nr` (the C++ routine `Rep_context::orientation_number`), and counts the *non-integral* positive roots that contribute, namely

- each non-integral positive *real* root α that is "oriented" with respect to γ (a parity condition on $\langle \gamma - \lambda + \rho_{\mathbb{R}}, \alpha^\vee \rangle$ together with the sign of $\langle \nu, \alpha^\vee \rangle$), and
- each non-integral complex quadruple $\{\pm\alpha, \pm\theta\alpha\}$ that is a complex descent (these are counted in pairs and then halved).

The essential point for us is that $\mathrm{or}(\gamma)$ depends on the continuous parameter ν: it is a property of where γ sits relative to the non-integral root hyperplanes, not merely of the block.

3 Stage (1): the c-form, and where orientation numbers live

c-form on a standard module: no orientation numbers

The c-form on a *standard* module $I(x)$ is computed purely by deformation in ν (`c_form_std` = `full_deform`). As $\nu \to 0$ one reaches the tempered case, where the c-form is positive; the deformation accumulates the controlled sign changes at reducibility points, encoded by Kazhdan–Lusztig–Vogan polynomials evaluated at $q = s$. No orientation number is involved in this step.

c-form on an irreducible module: orientation numbers enter here

To get the c-form on the irreducible $J(y)$ one inverts the Kazhdan–Lusztig–Vogan relationship. In the Grothendieck group,

$$J(y) = \sum_x (-1)^{\ell(x)-\ell(y)}\, P_{x,y}(1)\, I(x). \tag{1}$$

This identity of *characters* does *not* carry over verbatim to *forms*. At the level of c-forms one has instead

$$J(y)_c = \sum_x (-1)^{\mathrm{or}(x)-\mathrm{or}(y)}\, (-1)^{\ell(x)-\ell(y)}\, P_{x,y}(s)\, I(x)_c, \tag{2}$$

where two things change relative to the character formula:

- the polynomial is evaluated at $q = s$ rather than $q = 1$ (this is the signature refinement of the multiplicity), and
- each term acquires the **orientation-number sign** $(-1)^{\mathrm{or}(x)-\mathrm{or}(y)}$.

The difference $\mathrm{or}(x) - \mathrm{or}(y)$ is always even on terms that occur (an odd difference signals an error), so this sign is well defined; in the split ring it is the factor $s^{(\mathrm{or}(y)-\mathrm{or}(x))/2}$.

In the code this is exactly `c_form_irreducible`:

```
set c_form_irreducible (Param p) = KTypePol:
   let ori_nr_p = orientation_nr(p)
   then oriented_KL_sum = ParamPol: null_module(p) +
      for c@q in KL_sum_at_s(p) do (c*orientation_nr_term(ori_nr_p,q),q) od
in map(c_form_std@Param, oriented_KL_sum )
```

Here `KL_sum_at_s(p)` supplies $(-1)^{\ell(x)-\ell(y)} P_{x,y}(s)$, and

```
orientation_nr_term(orientation_nr(p),q) = s^{[or(p)-or(q)]/2}
```

supplies the orientation-number sign. Only after these standard-module coefficients are fixed up is each $I(x)_c$ replaced by its deformed value (`c_form_std`). The same factor appears at the built-in level as `exp_i(orient_y - orientation_number(x))` in `Rep_context`'s deformation routines.

Why it must be here. The c-form is canonically normalized on each *standard* module by the deformation (its sign is pinned down at the tempered endpoint). When one re-expresses $J(y)_c$ in that fixed basis of standard c-forms, the relative normalization between $J(y)$ and each $I(x)$ is precisely $(-1)^{\mathrm{or}(x)-\mathrm{or}(y)}$. Orientation numbers are therefore an intrinsic part of computing the c-form *on the irreducible*, even though they are absent from the c-form on a single standard module.

4 Stage (2): converting the c-form to the Hermitian form

The conversion is `convert_cform_hermitian`, and it does *not* use orientation numbers. It multiplies the c-form's value on each K-type p by s raised to $\mu(p) - \mu(p_0)$, where p_0 is a chosen normalizing K-type and

$$\mu(p) = \underbrace{(g - l - \rho_{\mathbb{R}}^{\vee}) \cdot (\lambda - \rho)}_{\lambda\text{-term}} + \underbrace{\tfrac{1}{2}\, l\, (\delta - 1)\, \tau}_{\tau\text{-term}} + \underbrace{\dim(\mathfrak{u} \cap \mathfrak{p})}_{\text{dimension term}} . \tag{3}$$

In code:

```
set convert_cform_hermitian (KTypePol P, mat delta, KType p0) = KTypePol:
  let a_mu = mu(p0,delta) in
    P.null_K_module +
      for c@p in P do (c*(mu(p,delta)-a_mu).rat_as_int.exp_s, p) od
```

The governing quantity here is μ — dominated by the parity of $\dim(\mathfrak{u}\cap\mathfrak{p})$ together with the lowest-K-type valuation — and not the orientation number. This is the only place the genuine real form (via δ and $\dim(\mathfrak{u} \cap \mathfrak{p})$) enters the sign bookkeeping; in the equal-rank case the twist is trivial and `hermitian_form_irreducible` is just `c_form_irreducible` followed by this conversion.

5 Conclusion

Step	atlas routine	orientation numbers?
c-form on standard $I(x)$	`c_form_std` / `full_deform`	no
c-form on irreducible $J(y)$	`c_form_irreducible`	**yes**
convert c-form $\to$ Hermitian	`convert_cform_hermitian`	no (uses μ)

Orientation numbers are part of **computing the c-form**, specifically the step that assembles the c-form on the irreducible from the c-forms on standard modules via formula (**??**). They are **not** the mechanism that converts the c-form into the Hermitian form; that role is played by μ and, in particular, by $\dim(\mathfrak{u} \cap \mathfrak{p})$.